Google A2A协议：Agent间通信标准

- 来源: https://class.imooc.com/lesson/5292#mid=105156
- 阶段: 阶段六: 【智能体】从单体到多体
- 周: 第12周 Multi-Agent：多智能体协作
- 模块: 多Agent协作：从个体到群体智能
- 课节: 图文 1-2 Google A2A协议：Agent间通信标准

1-2 Google A2A协议：Agent间通信标准

Google A2A协议：Agent间通信标准

> [!TIP] 阅读指南 在上一节中，我们探讨了多Agent协作的四种主要模式。但无论采用哪种协作模式，都有一个核心问题需要解决：Agent之间如何高效、可靠地通信？ 想象这样一个场景：你正在构建一个多Agent内容创作系统。Manager Agent需要将任务分配给Researcher Agent和Writer Agent，但它们使用不同的数据格式、通信协议，甚至运行在不同的平台上。如果没有统一的通信标准，每增加一个Agent，就需要为其与其他所有Agent建立专用的通信通道——这显然是不可扩展的。 这就是Google在2024年11月发布Agent-to-Agent Protocol（A2A协议）要解决的问题。A2A试图为Agent间通信建立类似HTTP之于Web的统一标准。

通信的核心挑战

在多Agent系统中，通信看似简单（不就是发消息吗？），实则面临三大核心挑战：

信息格式不统一：不同Agent可能使用不同的信息格式（JSON、YAML、自然语言等）。如果没有统一标准，每两个Agent之间都需要一个"翻译器"，N个Agent需要N×(N-1)个翻译器，这显然无法扩展。

上下文传递不完整：Agent间传递信息时需要平衡——传递太少可能导致错误决策，传递太多又会浪费Token资源。关键信息包括任务描述、当前进度、历史决策和相关背景。

异常处理机制缺失：当Agent处理失败时，缺乏标准化的错误通知、重试策略和任务转移机制，这会影响系统的可靠性和容错能力。

这些挑战都需要通过标准化的通信协议来解决。

A2A的三层设计

A2A协议借鉴了OSI网络协议的分层思想，把Agent通信拆成三层来解决：

为什么要分三层?

用快递类比：

- 第1层（数据模型层）：快递单上写什么内容（收件人、地址、物品）

第1层（数据模型层）

- 第2层（操作层）：快递公司能提供什么服务（寄件、查询、取消）

第2层（操作层）

- 第3层（协议绑定层）：用什么方式寄送（必须支持顺丰，可选支持京东、邮政）

第3层（协议绑定层）

关键设计思想：

关键设计思想

想象市面上有100个Agent，它们可能用不同语言（Python、Go、Java）、不同框架（LangChain、CrewAI、AutoGen）开发。如果没有统一标准：

- Agent A只会gRPC
- Agent B只会HTTP
- Agent C只会WebSocket → 它们无法互相通信!

它们无法互相通信!

A2A的解决方案：强制所有Agent至少支持JSON-RPC 2.0，这样保证了"最小公约数"——任意两个Agent都能通过JSON-RPC通信。在此基础上，你可以选择额外支持gRPC（追求性能）或HTTP/REST（方便Web集成），但这些是"锦上添花"。

举个实际例子：

举个实际例子

假设Agent A要委托Agent B写一篇文章：

```
第1层：构造数据
task = {
  "id": "task_123",
  "description": "写一篇关于AI的文章"
}

第2层：调用操作
agent_b.send_message(task, message)

第3层：选择协议（JSON-RPC必须支持）
通过JSON-RPC发送 → POST /rpc {...}
（如果Agent B还支持gRPC，也可以这样调用）
或通过gRPC发送 → A2AService.SendMessage(...)
或通过HTTP发送 → POST /tasks/task_123/messages {...}
```

三层分离让Agent可以跨平台、跨协议互操作——但JSON-RPC是所有Agent必须实现的"通用语言"。

A2A的核心协作模式

定义Agent之间如何配合完成任务：

1. Request-Response（请求-响应）：一问一答

用餐厅点菜类比：

- 你（Agent A）问服务员：“今天有什么特色菜？”
- 服务员（Agent B）回答：“有宫保鸡丁和鱼香肉丝”
- 特点：针对具体问题，得到具体答案，一轮对话结束

特点

2. Delegation（委托）：交付目标

用外包项目类比：

- 你（Agent A）对乙方说：“帮我设计一个企业官网，要求现代简洁”
- 乙方（Agent B）自己决定：用什么技术栈、几个页面、配色方案。...
- 完成后乙方主动交付成果

- 特点：只说目标，不管过程，对方全权负责

3. Collaboration（协作）：平等协商

用团队开会类比：

- 多个同事（Agent A、B、C）一起讨论方案
- 你提一个想法，我补充细节，他指出问题
- 反复交流直到达成共识
- 特点：平等关系，互相依赖，共同决策

核心区别：

核心区别

- Request-Response：问"怎么做"，得到答案
- Delegation：说"做什么"，等待成果
- Collaboration：大家一起"商量怎么做"

完整通信流程剖析

让我们通过一张完整的图，看看两个Agent基于A2A协议通信的全过程：

你可以清楚地看到：

发送请求（(1)(2)(3)(4)）：

发送请求（(1)(2)(3)(4)）

- Agent A: 应用层发起任务(1) → 消息层封装JSON → 传输层发送到队列(2)
- Agent B: 传输层从队列获取(3) → 消息层解析 → 应用层执行(4)

返回响应（(5)(6)(7)）：

返回响应（(5)(6)(7)）

- Agent B: 应用层生成结果 → 消息层封装 → 传输层发送到队列(5)
- Agent A: 传输层接收(6) → 消息层解析 → 应用层处理(7)

每一步都严格遵循三层架构，这就是A2A协议的优雅之处。

Agent Card：Agent的数字身份证

在多Agent系统中，有一个基础问题：Agent如何发现彼此并知道对方能做什么？

想象一个场景：你的系统里有10个Agent，新来的Agent想找一个能做"数据分析"的Agent协作。如果没有统一的发现机制，它只能逐个尝试，效率极低。

Agent Card是什么

Agent Card本质上是一个JSON格式的描述文档，就像Agent的"数字名片"。它包含了Agent的核心信息：

```
{
  "agentId": "agent_data_analyst",
  "name": "数据分析专家",
  "version": "1.0.0",
  "capabilities": [
    {
      "skill": "数据清洗",
      "inputFormat": ["csv", "json"],
      "outputFormat": ["cleaned_data"]
    },
    {
      "skill": "趋势分析",
      "inputFormat": ["time_series_data"],
      "outputFormat": ["trend_report", "visualization"]
    }
  ],
  "endpoint": "https://agent-service.com/analyst",
  "authentication": {
    "type": "API_KEY",
    "required": true
  },
  "metadata": {
    "description": "专业的数据分析Agent，擅长处理时间序列数据",
    "tags": ["数据", "分析", "可视化"]
  }
}
```

Agent Card的核心字段

- agentId：Agent的唯一标识

- capabilities：Agent能做什么，每个能力包含输入/输出格式
- endpoint：如何联系这个Agent
- authentication：需要什么样的认证方式
- metadata：描述性信息，便于搜索和理解

Agent发现机制

有了Agent Card这张"数字名片"，下一个问题是：Agent如何找到彼此？

在传统的微服务架构中，我们有服务注册中心（如Eureka、Consul）来管理服务发现。在多Agent系统中，也需要类似的机制——Agent注册中心。

发现流程如下：

发现流程如下

这个流程解决了几个关键问题：

- 动态发现：新Agent无需事先知道所有其他Agent的地址，通过注册中心动态查询
- 能力匹配：根据Agent Card的capabilities字段，精确找到具备所需能力的Agent
- 负载均衡：如果多个Agent具备相同能力，可以根据负载、地理位置等因素选择最优的

通过Agent Card和注册中心，系统实现了服务发现的能力——Agent可以动态地找到合作伙伴，而不需要硬编码依赖关系。这使得多Agent系统具备了真正的可扩展性。

错误处理：让系统更健壮

在分布式的多Agent系统中，错误是常态而非异常。网络可能中断、Agent可能崩溃、任务可能超时——系统必须优雅地处理这些情况。

标准化错误消息

A2A协议定义了统一的错误消息格式：

```
{
  "messageType": "error",
  "payload": {
    "errorCode": "TIMEOUT",
    "errorMessage": "Agent未在规定时间内响应",
    "originalMessageId": "msg_001",
    "timestamp": "2025-12-07T11:00:00Z",
    "retryable": true,
    "suggestedAction": "retry_after_5_seconds"
  }
}
```

常见错误类型

错误类型	含义	是否可重试
TIMEOUT	超时未响应	✓
AGENT_NOT_FOUND	目标Agent不存在	✗
INVALID_MESSAGE	消息格式错误	✗
CAPABILITY_NOT_SUPPORTED	Agent不支持该能力	✗
INTERNAL_ERROR	Agent内部错误	✓
RESOURCE_EXHAUSTED	资源耗尽（队列满等）	✓

A2A建议采用指数退避策略：

指数退避

> 什么是指数退避？ 指数退避是一种智能重试策略，核心思想是：每次重试的间隔时间按指数规律增长。 设计原理： 避免在网络拥塞或系统过载时，大量重试请求雪上加霜 给系统充分的恢复时间 逐步探测系统是否恢复正常

```
第1次重试：等待 1秒     # 初始试探
第2次重试：等待 2秒     # 稍作等待
第3次重试：等待 4秒     # 给系统更多恢复时间
第4次重试：等待 8秒     # 持续观察系统状态
...
```

这避免了在系统压力大时，大量重试进一步加重负担。

错误处理实例

```
// Manager收到Researcher的错误消息
{
  "messageType": "error",
  "sender": "agent_researcher",
  "receiver": "agent_manager",
  "payload": {
    "errorCode": "RESOURCE_EXHAUSTED",
    "errorMessage": "API调用超出配额限制",
    "retryable": true,
    "retryAfter": 3600  // 1小时后重试
  }
}

// Manager的处理策略
if (error.retryable && retryCount < 3) {
    // 等待指定时间后重试
    await sleep(error.retryAfter);
    retry();
} else {
    // 降级处理：使用备用Agent或返回部分结果
    fallback();
}
```

通过标准化的错误处理，系统可以自动应对各种异常情况，大大提升了可靠性。

A2A协议的价值

为什么需要像A2A这样的标准协议？

互操作性：不同厂商、不同框架开发的Agent可以互通。就像HTTP让不同的浏览器和服务器能通信一样。

可观测性：统一的消息格式便于监控、日志、调试。你可以清楚地看到每个Agent的工作状态、任务流转过程。

可扩展性：新增Agent只需遵循协议，无需修改现有Agent。系统可以从3个Agent扩展到30个、300个。

错误处理：标准化的错误消息格式，便于统一处理异常、重试、降级。

错误处理

当然，A2A协议还在演进中，很多细节仍在讨论。但其核心思想——通过标准化通信协议降低多Agent系统复杂度——已经被业界广泛认可。

下一节预告

理解了多Agent协作模式和通信标准，你可能迫不及待想动手实践了。下一节，我们将介绍专门为多Agent协作设计的CrewAI框架，带你从零开始构建完整的多Agent协作系统，包括团队编排、任务管理、能力增强等核心内容。

图片

